

Ali Project Installation Instructions

Generated from local Project Ali Markdown documentation.

Ali Project Installation Instructions

Generated: 2026-06-26

These instructions cover the current developer installation path. They are intentionally explicit so tomorrow's install pass can be verified step by step.

Required Dependencies

- Windows 11.
- PowerShell 5.1 or newer.
- .NET SDK 10.0.301 or compatible SDK for the repo.
- Visual Studio Community 2026 / Visual Studio 18 Community.
- Visual Studio workloads:
- .NET desktop development.
- Visual Studio extension development.
- ASP.NET and web development is useful for the WebHelper project.
- Git for Windows.
- Ollama installed locally if model-backed chat will be used.
- Current local proof model: qwen3-v1:8b.
- Optional Notepad++.
- Voice dependencies only when voice work resumes:
- Ali-owned lib\voice\python-venv.
- Faster-Whisper model cache.
- Piper executable and voice models.

Source Repo

Current developer repo:

C:\Users\clsor\Documents\Codex\2026-06-22\i-2

Current branch:

phase-1c-voice-hardening

Build

From the repo root:

```
dotnet build .\Ali.sln --no-restore -p:UseSharedCompilation=false -nr:false
```

Expected result:

```
Build succeeded.  
0 Warning(s)  
0 Error(s)
```

If the build reports locked WebHelper DLLs, stop only `Ali.App.WebHelper`, rebuild, and restart the helper afterward. Ollama does not need to be stopped for ordinary coding updates.

Test

From the repo root:

```
dotnet run --project .\tests\Ali.Tests\Ali.Tests.csproj --no-build
```

Expected result: all tests print PASS and the process exits with code 0.

Refresh DevRun

Current target:

```
%LOCALAPPDATA%\Ali\DevRun\Ali.App.Wpf.exe
```

Do not create new DevRun - * folders for this pass. Refresh the existing DevRun folder only after a clean build/test pass.

Start WebHelper

From the repo root:

```
dotnet run --project .\src\Ali.App.WebHelper\Ali.App.WebHelper.csproj --no-build
```

Default URL:

```
http://127.0.0.1:8765/
```

The WebHelper provides the browser companion and the loopback coding bridge used by the Visual Studio extension.

Install Visual Studio Extension

Build output:

```
src\Ali.App.VisualStudioExtension\bin\Debug\net472\Ali.App.VisualStudioExtension.vsix
```

Visual Studio Community installer path:

```
C:\Program Files\Microsoft Visual Studio\18\Community\Common7\IDE\VSIXInstaller.exe
```

Install/update command:

```
& "C:\Program Files\Microsoft Visual Studio\18\Community\Common7\IDE\VSIXInstaller.exe"  
/quiet ".\src\Ali.App.VisualStudioExtension\bin\Debug\net472\Ali.App.VisualStudioExtension
```

.vsix"

Open Visual Studio Community, then use:

View -> Ali Companion

or:

Tools -> Ali Companion

The extension is installed per Visual Studio instance. If Community is reinstalled later, reinstall the same VSIX into that new instance.

Runtime Model

Ali's current certified local proof path uses Ollama:

Endpoint: `http://127.0.0.1:11434/v1/`
Model: `qwen3-vl:8b`

Ali should not claim model-backed chat is active unless the runtime check passes.

Coding Companion Checks

After WebHelper and Visual Studio are running, verify these commands:

```
show visual studio integration
show packet commands
show packet ledger
resume build plan
plan package lookup Visual Studio tool window
preview project scaffold SolidWorks BOM helper
show windows troubleshooting toolkit
plan rogue process hunt port 8765
generate morning report
```

Safety Notes

- File edits still require preview and confirmation.
- Package installs, restore, build, test, and run commands still require confirmation.
- Git write commands still require confirmation.
- Windows troubleshooting commands are read-only guidance unless Chris explicitly approves a named repair action.
- Do not change signing, trust stores, Smart App Control, firewall, registry, global PATH, or service startup without explicit approval.